

CSCI-UA 102 - Makeup Quiz Solutions

Question and solution pairs

Makeup Quiz 1

Write a method `void oddSwap(PositionList<Integer> list)` that goes through every position of the given `PositionList` starting from the first. At every position, if the element at the position is even while the following element is odd, the elements are swapped. The implementation should run in linear time.

Solution.

Move through the positions once. After checking a pair, advance by one position, not by two positions. Swapping values with `set` keeps the positions themselves valid.

```
public void oddSwap(PositionList<Integer> list) {
    Position<Integer> p = list.first();
    while (p != null && list.after(p) != null) {
        Position<Integer> q = list.after(p);
        int a = p.getElement();
        int b = q.getElement();

        if (a % 2 == 0 && b % 2 != 0) {
            list.set(p, b);
            list.set(q, a);
        }
        p = list.after(p);
    }
}
```

The loop visits each position at most once, so the running time is $O(n)$.

Makeup Quiz 2

- (a) Implement `public static <E> boolean search(Stack<E> stack, E element)`. When the method returns, the stack must remain unchanged. Other stacks and queues may be initialized.
- (b) If push and pop are $O(1)$, what are the best and worst case asymptotic complexities?

Solution.

Pop items into an auxiliary stack until the element is found or the stack is exhausted. Then push everything back to restore the original order.

```
public static <E> boolean search(Stack<E> stack, E element) {
    Stack<E> temp = new LinkedStack<E>();
    boolean found = false;

    while (stack.size() > 0 && !found) {
        E current = stack.pop();
        temp.push(current);
        if (element == null ? current == null : element.equals(current)) {
            found = true;
        }
    }

    while (temp.size() > 0) {
```

```

        stack.push(temp.pop());
    }
    return found;
}

```

If the top element matches, only one pop and one push are needed, so the best case is $O(1)$. If the element is absent or at the bottom, every element is popped and restored, so the worst case is $O(n)$.

Makeup Quiz 3

Define $Y_1 = 2$, $Y_2 = 1$, $Y_3 = 3$, and $Y_{n+2} = Y_{n+1} + Y_n Y_{n-1}$. Write a recursive method to calculate the n -th term in linear time.

Solution.

For $k \geq 4$, the recurrence is $Y_k = Y_{k-1} + Y_{k-2}Y_{k-3}$. Carry the last three values in the recursive helper.

```

public static int Y(int n) {
    if (n <= 0) throw new IllegalArgumentException();
    if (n == 1) return 2;
    if (n == 2) return 1;
    if (n == 3) return 3;
    return YHelper(n, 3, 2, 1, 3);
}

private static int YHelper(int target, int index,
                           int yIndexMinus2,
                           int yIndexMinus1,
                           int yIndex) {
    if (index == target) return yIndex;
    int next = yIndex + yIndexMinus1 * yIndexMinus2;
    return YHelper(target, index + 1, yIndexMinus1, yIndex, next);
}

```

Each recursive call computes one new term, so the running time is $O(n)$.

Makeup Quiz 4

Implement a recursive method that counts how many times an element appears in a binary tree times the depth of each appearance:

```
public int depthCount(Tree<E> tree, E element).
```

For example, if the element appears twice at depths 1 and 4, then `depthCount` returns 5. Use depth as an auxiliary variable.

Solution.

The root has depth 0. At each node, add the current depth if the element matches, then recurse on all children with depth plus one.

```

public int depthCount(Tree<E> tree, E element) {
    if (tree.isEmpty()) return 0;
    return depthCount(tree, tree.root(), element, 0);
}

private int depthCount(Tree<E> tree, Position<E> p, E element, int depth) {

```

```

    int total = 0;
    E current = p.getElement();
    if (element == null ? current == null : element.equals(current)) {
        total += depth;
    }

    for (Position<E> child : tree.children(p)) {
        total += depthCount(tree, child, element, depth + 1);
    }
    return total;
}

```

The method visits each node once, so the running time is $O(n)$.

Makeup Quiz 5

Implement `public PositionList<E> appearOnce(PositionList<E> list)`, which returns a list of all elements in `list` that appear exactly once. Use `UnsortedMap`.

Solution.

First count each element using a map. Then scan the original list again and copy exactly-once elements into the answer, preserving their original order.

```

public PositionList<E> appearOnce(PositionList<E> list) {
    Map<E, Integer> counts = new UnsortedMap<E, Integer>();
    PositionList<E> result = new DoublyLinkedList<E>();

    for (E item : list) {
        Integer count = counts.get(item);
        if (count == null) {
            counts.put(item, 1);
        } else {
            counts.put(item, count + 1);
        }
    }

    for (E item : list) {
        if (counts.get(item) == 1) {
            result.addLast(item);
        }
    }
    return result;
}

```

With `UnsortedMap`, each map operation can scan the current map, so the worst-case time is $O(n^2)$. The extra space is $O(n)$.

Makeup Quiz 6

Given the `funnyList.equals` implementation in the prompt:

- What does `equals` check? Give two `funnyLists` that are equal by this method but not equal as ordinary `DoublyLinkedLists`.
- Based on this `equals`, implement a sensible `hashCode()`.

Solution.

The method checks three things:

- the two lists have the same size,
- all entries at odd indices 1, 3, 5, and so on are equal,
- the total sums of all entries are equal.

It does not require the even-index entries to match individually.

Example: [1, 5, 3] and [3, 5, 1] are equal by this method because they have the same size, the odd-index entry is 5 in both lists, and both sums are 9. They are not equal as ordinary lists because index 0 and index 2 differ.

A compatible hash code should use only information that `equals` requires to be the same: size, odd-index values, and total sum.

```
@Override
public int hashCode() {
    int result = 17;
    result = 31 * result + size();

    int sum = 0;
    for (int i = 0; i < size(); i++) {
        Integer value = getAtIndex(i);
        sum += value;
        if (i % 2 == 1) {
            result = 31 * result + value.hashCode();
        }
    }

    result = 31 * result + sum;
    return result;
}
```

If two lists are equal by the given `equals`, this hash code is the same for both.

Makeup Quiz 7

- Draw a binary search tree of entries with keys 1, 2, ..., 12 that has height 4.
- In what order should `put(key, value)` be called to get that tree?
- Remove entries in the order they were added. Redraw the tree after each removal.

Solution.

Tree notation used below: `x(L,R)` means a node with key `x`, left subtree `L`, and right subtree `R`. A dot, `.`, means an empty subtree. For example, `2(1,3(.,4))` means root 2, left child 1, right child 3, no left child of 3, and right child 4.

One valid height-4 BST is:

```
8(3(1(.,2),6(4(.,5),7)),10(9,11(.,12)))
```

This tree is obtained by inserting the keys in preorder:

```
8, 3, 1, 2, 6, 4, 5, 7, 10, 9, 11, 12
```

Using the course `BinarySearchTree.remove`, a node with two children is replaced by the largest key in its left subtree. Removing in the insertion order gives:

```
start:      8(3(1(.,2),6(4(.,5),7)),10(9,11(.,12)))
remove 8:   7(3(1(.,2),6(4(.,5),.)),10(9,11(.,12)))
remove 3:   7(2(1,6(4(.,5),.)),10(9,11(.,12)))
remove 1:   7(2(.,6(4(.,5),.)),10(9,11(.,12)))
remove 2:   7(6(4(.,5),.),10(9,11(.,12)))
```

```

remove 6: 7(4(.,5),10(9,11(.,12)))
remove 4: 7(5,10(9,11(.,12)))
remove 5: 7(.,10(9,11(.,12)))
remove 7: 10(9,11(.,12))
remove 10: 9(.,11(.,12))
remove 9: 11(.,12)
remove 11: 12
remove 12: .

```

Makeup Quiz 8

- (a) Draw two AVL trees that have the same elements but different topology. Describe insertion orders that produce each tree.
- (b) Add 8, 2, 14, 4, 12, 6, 10, 1, 15, 3, 13, 5, 11, 7, 9, 0 to an AVL tree. Redraw after each addition.
- (c) Remove those numbers in the same order. Redraw after each removal.

Solution.

The tree notation is the same as in Quiz 7: $x(L,R)$ means root x with left subtree L and right subtree R ; $.$ means empty.

For part (a), using keys 1, 2, 3, 4:

```

Insertion order 2, 1, 3, 4 gives: 2(1,3(.,4))
Insertion order 3, 4, 2, 1 gives: 3(2(1,.),4)

```

Both trees are valid AVL trees with the same keys and different topology.

For parts (b) and (c), the following is one valid AVL sequence. Deletion uses predecessor replacement for two-child nodes, matching the course BST deletion convention, followed by AVL rebalancing. Other valid AVL implementations can produce different but still balanced trees.

```

After additions:
add 8: 8
add 2: 8(2,.)
add 14: 8(2,14)
add 4: 8(2(.,4),14)
add 12: 8(2(.,4),14(12,))
add 6: 8(4(2,6),14(12,))
add 10: 8(4(2,6),12(10,14))
add 1: 8(4(2(1,.),6),12(10,14))
add 15: 8(4(2(1,.),6),12(10,14(.,15)))
add 3: 8(4(2(1,3),6),12(10,14(.,15)))
add 13: 8(4(2(1,3),6),12(10,14(13,15)))
add 5: 8(4(2(1,3),6(5,)),12(10,14(13,15)))
add 11: 8(4(2(1,3),6(5,)),12(10(.,11),14(13,15)))
add 7: 8(4(2(1,3),6(5,7)),12(10(.,11),14(13,15)))
add 9: 8(4(2(1,3),6(5,7)),12(10(9,11),14(13,15)))
add 0: 8(4(2(1(0,.),3),6(5,7)),12(10(9,11),14(13,15)))

```

```

After removals:
remove 8: 7(4(2(1(0,.),3),6(5,)),12(10(9,11),14(13,15)))
remove 2: 7(4(1(0,3),6(5,)),12(10(9,11),14(13,15)))
remove 14: 7(4(1(0,3),6(5,)),12(10(9,11),13(.,15)))
remove 4: 7(3(1(0,.),6(5,)),12(10(9,11),13(.,15)))
remove 12: 7(3(1(0,.),6(5,)),11(10(9,.),13(.,15)))
remove 6: 7(3(1(0,.),5),11(10(9,.),13(.,15)))
remove 10: 7(3(1(0,.),5),11(9,13(.,15)))
remove 1: 7(3(0,5),11(9,13(.,15)))
remove 15: 7(3(0,5),11(9,13))
remove 3: 7(0(.,5),11(9,13))
remove 13: 7(0(.,5),11(9,))
remove 5: 7(0,11(9,))
remove 11: 7(0,9)
remove 7: 0(.,9)
remove 9: 0
remove 0: .

```

Makeup Quiz 9

Write a method `int secondSmallest(int[] array)` that returns the second smallest element of the array in $O(n \log n)$ time.

Solution.

A direct $O(n \log n)$ solution is to sort and return the second item. This treats duplicate values as separate array entries. If the course expects the second distinct value, add a final scan for the first value larger than `array[0]`.

```
public static int secondSmallest(int[] array) {
    if (array.length < 2) throw new IllegalArgumentException();
    mergeSort(array, 0, array.length - 1);
    return array[1];
}

private static void mergeSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid + 1, hi);
    merge(a, lo, mid, hi);
}

private static void merge(int[] a, int lo, int mid, int hi) {
    int[] temp = new int[hi - lo + 1];
    int i = lo;
    int j = mid + 1;
    int k = 0;

    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= hi) temp[k++] = a[j++];

    for (int t = 0; t < temp.length; t++) {
        a[lo + t] = temp[t];
    }
}
```

Merge sort takes $O(n \log n)$, and returning index 1 takes $O(1)$.

Makeup Quiz 10

- (1) Write a method to calculate the depth of a node in a heap given its index: `public int depth(int index)`.
- (2) What positions in a heap can the second smallest element be in, in terms of height and size?
- (3) Implement `int secondMin()` that returns the second minimum entry in a heap of integers while checking as few elements as possible.

Solution.

For a 0-indexed array heap, the parent of index i is $(i - 1)/2$. The depth is the number of parent steps needed to reach index 0.

```

public int depth(int index) {
    if (index < 0 || index >= size) throw new IllegalArgumentException();
    int d = 0;
    while (index > 0) {
        index = (index - 1) / 2;
        d++;
    }
    return d;
}

```

In a min-heap, the smallest value is at the root. The second smallest value must be a child of the root, because every non-root node has a parent that is less than or equal to it. Therefore:

- if `size < 2`, there is no second minimum,
- if `size == 2`, it must be at index 1,
- if `size >= 3`, it must be at index 1 or index 2.

Those positions have depth 1.

```

public int secondMin() {
    if (size < 2) throw new IllegalArgumentException();
    if (size == 2) return array[1];
    if (array[1] <= array[2]) return array[1];
    return array[2];
}

```

This checks one element when `size == 2` and two child elements when `size >= 3`. The running time is $O(1)$.

Makeup Quiz 11

Write `public boolean pathExists(V a, V b)`, which returns true if there is a directed path from the vertex storing element `a` to the vertex storing element `b`. The graph is an `EdgeListGraph` with n nodes and m edges.

- Write a depth-first search method and give the efficiency.
- Write a breadth-first search method and give the efficiency.

Solution.

First find the vertices whose elements are `a` and `b`. Then run DFS or BFS along outgoing directed edges.

```

private Vertex<V> findVertex(V x) {
    for (Vertex<V> v : vertices()) {
        V value = v.getElement();
        if (x == null ? value == null : x.equals(value)) {
            return v;
        }
    }
    return null;
}

public boolean pathExistsDFS(V a, V b) {
    Vertex<V> start = findVertex(a);
    Vertex<V> target = findVertex(b);
    if (start == null || target == null) return false;

    Map<Vertex<V>, Object> seen = new HashMapSC<Vertex<V>, Object>();
    Stack<Vertex<V>> stack = new LinkedStack<Vertex<V>>();
}

```

```

stack.push(start);

while (stack.size() > 0) {
    Vertex<V> current = stack.pop();
    if (seen.get(current) == null) {
        if (current == target) return true;
        seen.put(current, 0);
        for (Edge<E, V> e : outgoingEdges(current)) {
            stack.push(e.getEndpoints()[1]);
        }
    }
}
return false;
}

```

For BFS, replace the stack with a queue.

```

public boolean pathExistsBFS(V a, V b) {
    Vertex<V> start = findVertex(a);
    Vertex<V> target = findVertex(b);
    if (start == null || target == null) return false;

    Map<Vertex<V>, Object> seen = new HashMapSC<Vertex<V>, Object>();
    Queue<Vertex<V>> queue = new LinkedList<Vertex<V>>();
    queue.enqueue(start);
    seen.put(start, 0);

    while (queue.size() > 0) {
        Vertex<V> current = queue.dequeue();
        if (current == target) return true;
        for (Edge<E, V> e : outgoingEdges(current)) {
            Vertex<V> next = e.getEndpoints()[1];
            if (seen.get(next) == null) {
                seen.put(next, 0);
                queue.enqueue(next);
            }
        }
    }
    return false;
}

```

In an `EdgeListGraph`, `outgoingEdges(v)` scans all m edges. DFS and BFS may call it for up to n vertices. Finding the start and target costs $O(n)$. Thus both methods have worst-case time $O(nm)$. With adjacency lists, the same algorithms would be $O(n + m)$, but not for this representation.

Makeup Quiz 12

We want `public Vertex<V> maxOutDegree()` for an `EdgeListGraph` with n nodes and m edges.

- One method loops through vertices and calls `outDegree` on every vertex. What is the worst-case asymptotic complexity?
- Write a more efficient implementation by initializing a `Map<Vertex<V>, Integer>` and looping through the edge list. What is the worst-case asymptotic complexity?

Solution.

For part (a), `outDegree(v)` calls `outgoingEdges(v)`, which scans all m edges in `EdgeListGraph`. Calling it for all n vertices costs $O(nm)$.

For part (b), to compute out-degree the map must count edges starting at each vertex. If the prompt says edges ending at every vertex, that would compute in-degree instead.

```
public Vertex<V> maxOutDegree() {
    Map<Vertex<V>, Integer> counts = new HashMapSC<Vertex<V>, Integer>();

    for (Vertex<V> v : vertices()) {
        counts.put(v, 0);
    }

    for (Edge<E, V> e : edges()) {
        Vertex<V> from = e.getEndpoints()[0];
        counts.put(from, counts.get(from) + 1);
    }

    Vertex<V> best = null;
    int bestCount = -1;
    for (Vertex<V> v : vertices()) {
        int count = counts.get(v);
        if (count > bestCount) {
            best = v;
            bestCount = count;
        }
    }
    return best;
}
```

Assuming expected $O(1)$ map operations, this runs in $O(n + m)$ time and uses $O(n)$ extra space.